Contents

Introduction

This document contains commonly used code for competitive programming, specifically in rust and written for the 2026 finale in the norwegian olympiad of informatics.

Basics

Vim config

Here is a basic vim config for competitive programming in rust:

```
let mapleader = " "
nnoremap <leader>ff :%!rustfmt<cr>
nnoremap <leader>fm :Explore<cr>

syntax on
set clipboard=unnamedplus
set mouse=a
set hlsearch
set incsearch
set ignorecase
set smartcase

set number
set relativenumber
set cursorline
set nowrap

set tabstop=4
set shiftwidth=4
set expandtab
set autoindent
set smartindent

hi CursorLine cterm=NONE ctermbg=236
hi MatchParen ctermfg=0 ctermbg=188
```

Place this in `~/.vimrc` at the start of the competition.

Rust boilerplate

```
use std::convert::TryInto;
use std::io;

fn main() {
  let mut lines = io::stdin().lines();
}
```

The program can then be compiled and run with

```
rustc -O -o main main.rs && ./main < 1.txt
```

Additionally, the rust code can be formatted with

```
rustfmt main.rs
```

Note

This is slightly more efficient than the above:

```
use std::io::{self, BufRead};

fn main() {
  let stdin = io:.stdin();
  let mut lines = stdin.lock().lines();
}
```

Capturing input

We will use the stdin lines iterator defined above to read the input:

```
let n: usize = lines.next().unwrap().unwrap().parse();
```

For reading multiple variables in a single line, one can do the following:

```
let [n, q]: [usize; 2] = lines
    .next()
    .unwrap()
    .unwrap()
    .trim()
    .split_whitespace()
    .map(|x| x.parse().unwrap())
    .collect::<Vec<_>>()
    .try_into()
    .unwrap();
```

, which is expandable to any number of values in the same line.

Note

This is unoptimal as it allocates a `Vec` when it is not necessary, but in my experience this was negligible. It can however be improved as such:

```
let line = lines.next().unwrap().unwrap()
let mut iter = line.trim().split_whitespace();
let n: usize = iter.next().unwrap().parse().unwrap();
let l: f32 = iter.next().unwrap().parse().unwrap();
let h: f32 = iter.next().unwrap().parse().unwrap();
```

This is useful for example when reading different types on the same line.

Reading a list of values

If the input is a single line split into values, it can be read as such:

```
let xs: Vec<usize> =
lines.next().unwrap().unwrap()..split_whitespace().map(|x|
x.parse().unwrap()).collect();
```

And if the input is given with each value on a separate line, it can be read with either of the following:

```
let xs: Vec<usize> = (0..n).map(|_|
lines.next().unwrap().unwrap().parse().unwrap()).collect();
```

```
let mut xs: Vec<usize> = Vec::new();
for _ in 0..n {
  let x = lines.next().unwrap().unwrap().parse().unwrap();
  xs.push(x);
}
```

```
let mut xs: Vec<usize> = vec![usize::MAX; n];
for i in 0..n {
  xs[i] = lines.next().unwrap().unwrap().parse().unwrap();
}
```

I prefer the first of the three, but the others can be useful if the line contains more complex input.

Interactive programs

A general theme across interactive problems is to manipulate the mathematical equation used so that all the local variables are extracted, such that we can precompute the values to be used for each iteration of the program. Input and output are handled the same way as in programs with static input.

Memory Usage

This table is an overview of memory usage on the stack by different types. Note that some types such as `&str` take 16 bytes on the stack, and contain a pointer to more data on the heap.

Type	Bytes
`i8`, `u8`	1
`i16`, `u16`	2
`i32`, `u32`	4
`i64`, `u64`	8
`i128`, `u128`	16
`isize`, `usize`	8
`f32`	4
`f64`	8
`bool`	1
`char`	4
`&T`	8
`*const T`, `*mut T`	8
`fn()`	8
`&[T]`	16
`&str`	16
`Vec<T>`	24
`String`	24

Strings and chars

Parsing a number in a string can be done like this:

```
let x: usize = s.parse().unwrap();
let x: i32 = s.parse().unwrap();
```

And with chars, as such:

```
let s: char = 'a';
let x: u32 = s.to_digit(10);
let x: u8 = s.to_digit(10) as u8;

let s: &str = "123";
let x: Vec<u32> = s.chars().map(|x| x.to_digit(10)).collect();
```

One can get an iterator of chars from a string by using `.chars()`. An alternative and more efficient way to get the value as a digit is by converting it to u8 to get its ascii value.

```
let c: char = '3';
let n: u8 = c as u8 - b'0';
let n: usize = (c as u8 - b'0') as usize;
```

Note that this only works when $x \in [0, 9]$.

String and &str

`String` is an owned and heap-allocated string type. This means that it can grow and shrink.

```rust
let mut s: String = String::from("Hello");
s.push_back(", world!");

println("{}", s);
```

On the other hand, `&str` refers to a string slice. This is a borrowed reference, which is usually immutable.

```rust
let s: String = Strong::from("hello");
let slice: &str = s[0..2]; // "he"
```

It is possible to convert between them:

```rust
let s: String = String::from("hello");
let slice: &str = &s;
let slice: &str = s.as_str();

let slice: &str = "hello";
let s = slice.to_string();
let s = String::from(slice);
```

`io::stdin().lines().next().unwrap().unwrap()` returns a `String`.

Bitmasks and bitwise operations

Operator	Operation
`&`	AND
`|`	OR
`^`	XOR
`!`	NOT
`<<`	Left shift
`>>`	Right shift

Bitmasks are useful when one needs to store many related booleans in a way which makes it easier to run operations on multiple of them at the same time. It is usually easiest to use this if the number of booleans is less than 64, which means that it can fit into an `u64`. Here is an example usage of bitmasks to handle the door permissions in `nio23-finale-noekkelkort`:

```rust
use std::collections::{HashSet, VecDeque};
use std::convert::TryInto;
use std::io;

fn main() {
    let mut lines = io::stdin().lines();

    let [n, m, k, t]: [usize; 4];

    let all_permissions: u32 = (1 << k) - 1;

    // graph[from] = [(to, permissions)]
    let mut graph: Vec<Vec<(usize, u32)>> = vec![Vec::new();
n];
    // edges in the graph to check
    let mut pairs: Vec<(usize, usize, u32)> = Vec::new();

    for &(u, v, p) in &pairs {
        let mut queue: VecDeque<usize> = VecDeque::new();
        let mut visited: HashSet<usize> = HashSet::new();
        let permissions = !p & all_permissions;
        queue.push_back(u);

        let mut result = true;

        // Note that this is an inefficient implementation of
BFS
        // It would be better to check if it is visited
*before* pushing
        while let Some(a) = queue.pop_front() {
            if visited.contains(&a) {
                continue;
            }
            visited.insert(a);

            if a == v {
                result = false;
                break;
            }

            for &(n, ps) in &graph[a] {
                if permissions & ps != 0 {
                    queue.push_back(n);
                }
            }
        }

        println!("{}", result as u8);
    }
}
```

Useful Bit Hacks

Check whether a number is a power of 2:

```rust
x != 0 && (x & (x - 1)) == 0;
```

Get lowest set bit:

```rust
x & (!x + 1);
// Alternatively:
x & x.wrapping_neg();
```

Remove the lowest set bit:

```rust
x &= x - 1;
```

Count number of set bits (popcount):

```rust
x.count_ones();
```

Get index of lowest set bit:

```rust
x.trailing_zeros();
```

Get index of highest set bit (for u64):

```rust
63 - x.leading_zeros();
```

Iterate over all subsets of a bitmask

```rust
let mut sub = mask;
while sub > 0 {
    sub = (sub - 1) & mask;
}
```

Iterate over set bits

```rust
let mut x = mask;

while x > 0 {
    let bit = x & (!x + 1);
    let idx = bit.trailing_zeros();
    // Do something with the idx
    x ^= bit;
}
```

Next combination with same popcount

```
let u = x & (!x + 1);
let v = x + u;
v + (((v ^ x) / u) >> 2)
// Example: 00111 → 01011
```

Check if `a` is a subset of `b`:

```
(a & b) == a;
```

Toggle bits:

```
x ^= 1 << k; // Toggle bit
x |= 1 << k; // Set bit
x &= !(1 << k); // Clear bit
(k >> x) & 1; // Check bit
```

Iterate over all masks of size `n`:

```
for mask in 0..(1 << n) { }
```

Match

Useful for pattern matching. Works similar to that of languages like haskell or python. It can contain expressions and also return values directly.

```
let bar = match foo {
    Some(x) = x,
    None = 0,
}
```

```
match foo {
    Some(0) = {
        println!("Value is zero :o");
    },
    Some(x) if x < 10 {
        println!("X is less than 10");
    },
    Some(x) {
        println!("Value: {}", x);
    },
    None = {
        println!("No value :(");
    }
}
```

Note

One can use the operator `@` to set a value while matching a pattern. For example, one can combine the pattern `x` and `1..=10` into a single expression with `x @ 1..=10`, which will keep the original value in `x` while matching it against the pattern.

Integer overflow

In problems where large numbers are required, for example finding the number of unique nonempty subsequences, the number grows exponentially and thus we need to clamp it to some number. Usually this number is defined as $MOD = 1\,000\,000\,007$ (because this is a prime number). In the aforementioned problem, we can then use the following to ensure the number stays within the bounds of a 64-bit integer.

```
const MOD: usize = 1_000_000_007;
let new_subsequences = (subsequences * 2 - last[x as usize] +
MOD) % MOD;
```

Time complexity

Depending on the constraints in the problem, different algorithms should be chosen, approximately based on this:

n	Time complexity
1 000 000	$O(n)$
400 000	$O(n \log n)$
1 000	$O(n^2)$
200	$O(n^3)$
20	$O(2^n)$
10	$O(n!)$

Useful data types

Option

An option holds an optional value, similar to the `Maybe`-monad in haskell:

```
let x: Option<i32> = None;
let x: Option<i32> = Some(42);

match x {
  Some(n) => println!("value exists"),
  None => println!("no value"),
}

let x: i32 = x.unwrap(); // unsafe; panics if None
let x: i32 = x.unwrap_or(0); // safe; adds a fallback value
let x: Option<i32> = x.map(|v| v * 2);

if let Some(a) = x {
  println!("{}", a);
}

if let Some(v) = x {
  if v == 42 {
    println!("X is some and 42!")
  }
}

// Equivalent to above, but might not work in the version of
rust used by NIO
if x.is_some_and(|v| v == 42) {}
if let Some(v) = x && v == 42 {}
```

Here is an example, taken from my solution to `nio24-finale-sokkeskuff`:

```
let mut prev: Option<usize> = None;
let mut pairs = 0;

for &s in &socks {
    if let Some(p) = prev {
        if s - p <= t {
            pairs += 1;
            prev = None;
            continue;
        }
    }
    prev = Some(s);
}
```

Enums

Can be defined with

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum Instruction {
    Build,
    Query,
}
```

Vectors

Vectors are lists of items with a dynamic length. Here are some examples:

```
let xs: Vec<i32> = vec![1, 2, 3, 4];
let xs: Vec<i32> = vec![0; 5]; // [0, 0, 0, 0, 0]
let mut xs: Vec<usize> = Vec::new(); // xs = vec![]
xs.push(42); // xs = vec![42]
xs.push(123); // xs = vec![42, 123]
```

VecDeque

For algorithms such as 0-1 BFS, one can use a double-ended queue as follows:

```
use std::collections::VecDeque;

let mut queue: VecDeque<usize> = VecDeque::new();

while let Some(x) = queue.pop_front() {
  if weights[x] == 0 {
    for &n in &graph[x] {
      queue.push_front(n);
    }
  } else {
    for &n in &neighbors[x] {
      queue.push_back(n);
    }
  }
}
```

Note that this is just pseudocode.

Slices

A vector can be indexed with a slice as such:

```
let xs = vec![0, 1, 2, 3, 4];

let slice = &xs[1..4]; // [1, 2, 3]
let slice = &xs[1..=3]; // [1, 2, 3]
let slice = &xs[..3]; // [0, 1, 2]
let slice = &xs[2..]; // [2, 3, 4]
let slice = &xs[..]; // [0, 1, 2, 3, 4]
```

Slices can also be mutable:

```
let xs = vec![0, 1, 2, 3, 4];

for x in &mut xs[1..4] {
  *x *= 2;
}

println!("{:?}", xs); // [0, 2, 4, 6, 4]
```

Note

Slices borrow from the vector. `xs[1..4]` produces a slice of type `&[i32]`.

A slice can also be used as an iterator, in pattern matching, etc.

```
let xs = vec![0, 1, 2, 3, 4];

match &xs[..] {
    [first, .., last] => {
        println!("First: {}, Last: {}", first, last);
    }
}

// .iter() on a slice of type &[i32] creates an iterator of
references (`&i32`)
let slice_sum = xs[1..4].iter().sum(); // 6
```

Sets and maps

HashSet

Note that most set operations use borrows, with the exception of `insert`.

```
use std::collections::HashSet;

let mut set: HashSet<usize> = HashSet::new();

set.insert(5); // true
set.insert(6); // true
set.insert(6); // false (already exists)

assert!(set.contains(&5));
assert!(!set.contains(&7));

set.remove(&6);
```

Note

When doing a lookup, one uses a reference to the value, not the value itself (`set.contains(&5)`). The same applies to HashMaps.

HashSets also have some useful in-place functions

```
let mut set: HashSet<usize> = HashSet::from([1, 2, 3, 4, 5,
6]);

set.retain(|x| x % 2 == 0);
dbg!(&set); // { 2, 4, 6 }

let set_b: HashSet<usize> = HashSet::from([7, 8, 9]);
set.extend(set_b);
dbg!(&set); // { 2, 4, 6, 7, 8, 9 }
```

HashMap

HashMaps store data in a map with hashed keys and have $O(1)$ operations.

```
use std::collections::HashMap;

let mut map: HashMap<usize, &str> = HashMap::new();
map.insert(1, "one");
map.insert(2, "two");

let x: Option<&str> = map.get(&1); // Some("one")
let x: Option<&str> = map.get(&3); // None

assert!(map.contains_key(&2));
map.remove(&2);
assert!(!map.contains_key(&2));

map.insert(1, "en");
```

There is also more complex funcionality present, such as:

```
let mut map: HashMap<usize, i32> = HashMap::new();
map.insert(1, 123);

let x: &i32 = map[&1]; // unsafe; panics if 1 does not exist
let x: Option<&i32> = map.get(&1); // safe

let mut x: &mut i32 = map.entry(1); // .entry() returns an
&mut i32
let x: &i32 = map.entry(1).or_insert(456); // 123
let x: &i32 = map.entry(2).or_insert(456); // 456

*map.entry(3).or_insert(0) += 1; // map[&3] == 1
*map.entry(3).or_insert(0) += 1; // map[&3] == 2

map.entry(4).and_modify(|ref mut v| v += 1).or_insert(1); //
map[&4] = 1
```

```
map.entry(4).and_modify(|v| *v += 1).or_insert(1); // map[&4]
= 2
```

BTreeSet and BTreeMap

BTreeSet and BTreeMap work conceptually same as their Hash counterparts, with the addition that they are automatically sorted. The difference is that the Hash versions are (as the name suggests) hash-based with $O(1)$ operations, and the BTree versions have $O(\log n)$ operations..

Note

These data types internally use a B-tree to sort the values, hence the name “B-Tree Set”.

Here are some usage examples:

```
use std::collections::BTreeSet;

let mut set = BTreeSet::new();
set.insert(10);
set.insert(5);
set.insert(20);

let left: Option<&i32> = set.range(..10).next_back();
let right: Option<&i32> = set.range(10 + 1..).next();
```

If one needs to store data along with the sorted indexes, a BTreeMap is useful. It would also work to use a separate HashMap and BTreeSet, but combining it into a BTreeMap is more efficient and makes the code cleaner.

```
use std::collections::BTreeMap;

let mut map = BTreeMap::new();
map.insert(10, "root");
map.insert(5, "left child");
map.insert(20, "right child");

let left_depth: Option<&&str> =
map.range(..10).next_back().map(|(_, v)| v);
// Optionally:
let left_depth: Option<&str> =
map.range(..10).next_back().map(|(_, v)| *v);
```

Data structures

Graphs

I like to define graphs as follows:

```
type Graph = Vec<Vec<(usize, usize)>>;
```

Which will be indexed like this:

```
graph[from_node] = [(neighbor1, cost), (neigbor2, cost),
(neighbor3, cost)]
```

Note

Unweighted graphs can be simplified to

```
type Graph = Vec<Vec<usize>>;
```

Here is an example of how a graph can be constructed in rust (taken from my solution to `nio21-finale-togtur`). In this case I am creating an undirected graph. For a directed graph, one would remove the line with `graph[b].push((a, k))`.

```
let mut graph: Vec<Vec<(usize, usize)>> = vec![Vec::new(); n];
for _ in 0..m {
    let [a, b, k]: [usize; 3] = lines
        .next()
        .unwrap()
        .unwrap()
        .trim()
        .split_whitespace()
        .map(|x| x.parse().unwrap())
        .collect::<Vec<_>>()
        .try_into()
        .unwrap();
    graph[a].push((b, k));
    graph[b].push((a, k));
}
```

Dijkstra

Dijkstra’s algorithm is an algorithm for finding the lowest total weight path between two nodes in a graph. It has a time complexity of $O((V + E) \log V)$. It requires the following imports:

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;
```

Note

We use `Reverse` here because rust’s `BinaryHeap` sorts the values descending, but for Dijkstra we want it to be ascending. `Reverse` is used to reverse the natural ordering of values.

And can be implemented as such:

```
let n: usize; // Number of nodes in the graph
let graph: Graph;
let start: usize;
let target: usize;

let mut heap = BinaryHeap::new();
heap.push(Reverse((0, start)));
let mut dist = vec![usize::MAX; n];
dist[start] = 0;

let mut result = None;

while let Some(Reverse((cost, node))) = heap.pop() {
    if cost > dist[node] {
        continue;
    }

    if node == target {
        result = Some(cost);
        break;
    }

    for &(neighbor, c) in &graph[node] {
        let new_cost = cost + c;
        if new_cost < dist[neighbor] {
            dist[neighbor] = new_cost;
            heap.push(Reverse((new_cost, neighbor)));
        }
    }
}

println!("{}", result.unwrap());
```

There are often problems which require a variation of Dijkstra, for example the aforementioned `nio21-finale-togtur` which adds another layer for each “free edge”. This can easily be implemented by extending the dist DP table and adding another item to the tuple in the heap:

```
heap.push(Reverse((0, v, start))); // (cost, tickets, node)
let mut dist = vec![vec![usize::MAX; v + 1]; n];
dist[start][v] = 0;

// ---

for &(n, c) in &graph[node] {
    let new_cost = cost + c;
    if new_cost < dist[n][tickets] {
        dist[n][tickets] = new_cost;
        heap.push(Reverse((new_cost, tickets, n)));
    }

    if tickets > 0 && cost < dist[n][tickets - 1] {
        dist[n][tickets - 1] = cost;
        heap.push(Reverse((cost, tickets - 1, n)));
    }
}
```

BFS / DFS

BFS (breadth-first search) and DFS (depth-first search) are both algorithms used to traverse an unweighted graph. They both use a queue, with the difference being that BFS uses a FIFO-queue while DFS uses a LIFO-stack. They both have an asymptotic time complexity of $O(V+E)$. Here is a simple example implementation of BFS in rust which returns a boolean for whether or not a path exists:

```
use std::collections::{HashSet, VecDeque};

let graph: Vec<Vec<usize>>;
let start: usize;
let target: usize;

let mut queue: VecDeque<usize> = VecDeque::new();
let mut visited: HashSet<usize> = HashSet::new();
queue.push_back(start);
visited.insert(start);

let mut result = false;

while let Some(a) = queue.pop_front() {
  if a == target {
    result = true;
    break;
  }

  for &n in &graph[a] {
    if !visited.contains(&n) {
      queue.push_back(n);
      visited.insert(n);
    }
  }
}
```

This pushes to the back and pops from the front, thus making it a breadth-first search in which all neighbor nodes are searched before continuing to the next level. The only difference is changing this line:

```
- queue.push_back(n);
+ queue.push_front(n);
```

BFS is often the best when trying to find the shortest path between two nodes, while DFS is preferred if all we need is to know whether or not a path exists. Finding the path length with BFS can be done as such:

```
use std::collections::{HashSet, VecDeque};

let graph: Vec<Vec<usize>>;
let start: usize;
let target: usize;

let mut queue: VecDeque<(usize, usize)> = VecDeque::new();
let mut visited: HashSet<usize> = HashSet::new();
queue.push_back((start, 0));
visited.insert(start);

let mut result = None;

while let Some((a, c)) = queue.pop_front() {
  if a == target {
    result = Some(c);
    break;
  }

  for &n in &graph[a] {
    if !visited.contains(&n) {
      queue.push_back((n, c+1));
      visited.insert(n);
    }
  }
}
```

0-1 BFS

0-1 BFS is an extension of BFS in which a double ended queue is used to determine which nodes to traverse first. This is used for a special case of weighted graphs where the weights are all either 0 or 1.

```
use std::collections::VecDeque;

let n: usize; // n of nodes
let graph: Vec<Vec<(usize, usize)>>; // (target, weight)
let start: usize;
let target: usize;

let mut queue: VecDeque<usize> = VecDeque::new();
let mut dist: Vec<usize> = vec![usize::MAX; n];
queue.push_back(start);

while let Some(a) = queue.pop_front() {
  for &(n, c) in &graph[a] {
    let new_dist = dist[a] + c;
    if new_dist < dist[n] {
      dist[n] = new_dist;
      if c == 0 {
        // Prioritize weight 0
        queue.push_front(n);
      } else {
        queue.push_back(n);
      }
    }
  }
}
```

Bellman-Ford

The time complexity of this algorithm is $O(V \cdot E)$. The algorithm works by iterating through all edges `n-1` times, and relaxing the edges if possible. The benefit to this compared to Dijkstra's algorithm is that it works with negative edge weights, while Dijkstra does not. In addition, this has the ability to detect negative cycles, which is a cycle with a negative total weight such that there does not exist any shortest path, as the weight will just approach $-\infty$. Here is a simple implementation in rust:

```
let n: usize;
let edges: Vec<(usize, usize, i32)>; // [(from, to, weight)]

let inf = i32::MAX / 2;
let mut dist = vec![inf; n];
let source = 0;
dist[source] = 0;

for _ in 0..n - 1 {
    let mut changed = false; // Optional optimization
    for &(u, v, w) in &edges {
        if dist[u] != inf && dist[u] + w < dist[v] {
            dist[v] = dist[u] + w;
            changed = true;
        }
    }
    if !changed {
```

```
        break;
    }
}
```

In addition, negative cycles can be detected by relaxing the edges one more time:

```
for &(u, v, w) in &edges {
    if dist[u] != inf && dist[u] + w < dist[v] {
        println!("Negative cycle found!");
        break;
    }
}
```

Floyd-Warshall

The Floyd-Warshall algorithm is an algorithm for finding the all the shortest paths between any two nodes in a directed or undirected graph.

```
let n: usize;
// graph[from] = [(weight, to)];
let graph: Vec<Vec<(i32, usize)>>;

let mut d = vec![vec![i32::MAX / 2; n]; n];
for i in 0..n {
    d[i][i] = 0;
    for &(w, j) in &graph[i] {
        d[i][j] = w;
    }
}
for k in 0..n {
    for i in 0..n {
        for j in 0..n {
            d[i][j] = d[i][j].min(d[i][k] + d[k][j]);
        }
    }
}
```

This algorithm is $O(V^3)$, so it is very slow for finding a single path compared to Dijkstra's algorithm, but it will be much faster if one needs to find the minimum total weight of all paths in the graph.

Strongly Connected Components

In a directed graph, a strongly connected component is a maximal group of vertices such that for every pair of vertices u and v in the group, there is a path from $u \to v$ and $v \to u$. This means that every node can reach every other node following the direction of the edges.

Note

Here is an example:

```
1 → 2 → 3
↑   ↓
5 ← 4
```

In this case, $\{1, 2, 4, 5\}$ form one SCC. 3 can not reach back to the others, so it forms its own SCC.

If the SCC is compressed into a single node, this becomes a Directed Acyclic Graph (DAG). It is also useful for detecting cycles in directed graphs.

Kosaraju's algorithm

When the SCC has more than one node, this means that there exists a cycle. We can use Kosaraju's algorithm to take a directed graph and create the SCC from this:

```
// Initialize the graph
let n = 4;
let mut graph = vec![Vec::new(); n];
graph[0].push(1);
graph[1].push(2);
graph[2].push(0);
graph[2].push(3);

// Create a reversed graph
let mut rg = vec![Vec::new(); n];
for v in 0..n {
    for &to in &graph[v] {
        rg[to].push(v);
    }
}

// First DFS pass. This determines the order to process the
nodes
// so SCCs are discovered correctly in the second pass of DFS
let mut visited = vec![false; n];
let mut order = vec![];
let mut stack = vec![];
for i in 0..n {
    // For each unvisited node, do a DFS starting from `i`
    // This is used to build a "finishing order"-stack
    if visited[i] {
        continue;
    }
    stack.push((i, 0));
    while let Some((v, state)) = stack.pop() {
        if state == 0 {
            if visited[v] {
                continue;
            }
            visited[v] = true;
            stack.push((v, 1));
            for &to in &graph[v] {
                if !visited[to] {
                    stack.push((to, 0));
                }
            }
        } else {
            order.push(v);
        }
    }
}

// The second DFS explores backward connections. This ensures
that the
// DFS starting from the last finished node in the first pass
will
// only stay inside of the SCC.
let mut component = vec![usize::MAX; n];
let mut id = 0; // The number of components
// As described, this iterates from the reverse finishing
order.
for &v in order.iter().rev() {
    if component[v] != usize::MAX {
        continue;
    }
    // Run a DFS for each node which is not a member of an SCC
    let mut stack = vec![v];
    component[v] = id;
    while let Some(x) = stack.pop() {
        for &to in &rg[x] {
            if component[to] == usize::MAX {
                component[to] = id;
                stack.push(to);
            }
        }
    }

    id += 1;
}

// Each node will now have a component ID
dbg!(&id, &component);
```

This algorithm runs DFS twice, and as such it has a time complexity of $O(V + E)$.

Building a DAG

When building a DAG from an SCC, each component becomes a single node. For every edge $u \to v$ in the original graph, if `component[u] != component[v]`, it is an edge between two SCCs

and will add an edge between them in the SCC. The above algorithm can be extended as follows:

```
// Create the dag
let mut dag = vec![Vec::new(); id];
for u in 0..n {
    for &v in &graph[u] {
        let cu = component[u];
        let cv = component[v];
        if cu != cv {
            dag[cu].push(cv);
        }
    }
}

// Optional deduplication (often not necessary)
for edges in dag.iter_mut() {
    edges.sort_unstable();
    edges.dedup();
}

dbg!(&dag);
```

2-SAT

2-Satisfiability is a type of boolean problem where one has n boolean variables and there are clauses of the form $a \vee b$, for example:

```
(x0 or x1)
(!x0 or x2)
(x1 or !x2)
```

We want to find out if it is possible to assign `true` / `false`-variables to satisfy all clauses. One can convert the clause $a \vee b$ into two implications: $a \vee b \equiv (\neg a \Rightarrow b) \vee (\neg b \Rightarrow a)$. This gives two edges in a directed graph. Thus, we can create a graph where each variable corresponds to two nodes: itself and its negation, connected with these implications as edges. We can then run SCC on this graph. For 2-SAT to be satisfiable, no variable x can appear in the same SCC as its negation $\neg x$.

```
// Initialize the graph
let variables = 3;
let n = variables * 2;
let mut graph = vec![Vec::new(); n];

let add_or = |graph: &mut Vec<Vec<usize>>, a: usize, b: usize|
{
    graph[a ^ 1].push(b);
    graph[b ^ 1].push(a);
};

let x0 = 0;
let nx0 = 1;
let x1 = 2;
let nx1 = 3;
let x2 = 4;
let nx2 = 5;

add_or(&mut graph, x0, x1); // x0 V x1
add_or(&mut graph, nx0, x2); // !x0 V x2
add_or(&mut graph, nx1, nx2); // !x1 V !x2

// --- Kosaraju SCC from above ---

let mut ok = true;
let mut assignment = vec![false; variables];
for i in 0..variables {
    if component[2 * i] == component[2 * i + 1] {
        // x_i and !x_i were found in the same SCC
        ok = false;
        break;
    } else {
        // True if x_i's SCC comes after !x_i's SCC
        assignment[i] = component[2 * i] > component[2 * i +
1];
    }
}

if ok {
    println!("Satisfiable with assignment {:?}", assignment);
} else {
    println!("Unsatisfiable :(");
}
```

Note

The graph has to have twice as many nodes as there are variables. Each variable has the true value at $2i$ and the false value at $2i + 1$.

Topological sort

A topological sort is an ordering of nodes in a directed acyclic graph (DAG) such that for every directed edge $u \to v$, node u comes before node v in the ordering. Note that this is only possible for acyclic graphs.

```
use std::collections::VecDeque;

let mut visited = vec![false; n];
let mut order = VecDeque::new();

for start in 0..n {
    if visited[start] {
        continue;
    }

    let mut stack = Vec::new();
    stack.push((start, false));

    while let Some((node, children_visited)) = stack.pop() {
        if children_visited {
            order.push_front(node);
            continue;
        }

        if visited[node] {
            continue;
        }

        visited[node] = true;
        stack.push((node, true));

        for &neighbor in graph[node].iter().rev() {
            if !visited[neighbor] {
                stack.push((neighbor, false));
            }
        }
    }
}
```

Now, `order` will be a `VecDeque` with the correct ordering of the nodes.

Minimum Spanning Tree

A minimum spanning tree (MST) in a connected, weighted and undirected graph is a subset of the edges which

- Connects all vertices
- Has no cycles (and is thus a tree)
- Contains excactly $V - 1$ edges if there are V vertices.

Kruskal's algorithm for MST

Kruskal's algorithm is an algorithm used to find the total weight of the minimum spanning tree. This uses the DSU-implementation also found in this document.

```
// Given a list of edges with (weight, from, to)
let edges: Vec<(i32, usize, usize)>;

edges.sort_by_key(|&(w, _, _)| w); // Sort by weight ascending
let mut dsu = DSU::new(n);
let mut mst_weight = 0;
```

```
for (w, u, v) in edges {
    if dsu.union(u, v) {
        mst_weight += w;
    }
}
```

Dynamic Programming / DP

Dynamic Programming (DP) is a technique to solve problems by breaking them into overlapping subproblems and storing their results to avoid recomputation. DP is applicable to a problem if it satisfies the following:

- Optimal substructure: the optimal solution to the problem can be constructed from the optimal solutions of its subproblems.
- Overlapping subproblems: the same subproblems are solved multiple times and can be “cached”. DP is unnecessary if independent subproblems are computed only once.

Usually, the problem can be expressed as a sequence of choices, such as in knapsack. There exist two main types of DP: memoization (top-down) and tabulation (bottom-up). It is usually possible to apply both to the same problem, but i usually prefer tabulation as it avoids recursion.

Here is an example of a DP problem in which I want to compute the nth fibonacci number. First, this is a solution using memoization:

```
use std::collections::HashMap;

fn fibonacci(n: u64, memo: &mut HashMap<u64, u64>) -> u64 {
  if n <= 1 {
    return n;
  }

  if !memo.contains(&n) {
    let value = fibonacci(n-1, memo) + fibonacci(n-2, memo);
    memo.insert(n, value);
  }

  *memo.get(&n).unwrap()
}
```

And here is the same problem, but with tabulation:

```
fn fibonacci(n: usize) -> u64 {
  if n <= 1 {
    return n as u64;
  }

  let mut dp = vec![u64::MAX; n + 1];
  dp[0] = 1;
  dp[1] = 1;
  for i in 2..=n {
    dp[i] = dp[i-1] + dp[i-2];
  }

  dp[n]
}
```

Tabulation is usually a bit harder to implement than memoization, as it can often be applied by simply adding a memo table to memoize the outputs of a function, but tabulation can be more powerful and straightforward.

If a problem looks like DP can be applied, it is usually best to first sketch the DP state and transition before trying to implement it. Here are a few examples of using DP. The common thing between them is that they all rely on making a choice for each item, and using this to define the DP.

In NIO problems, DP is also often used when there is a problem in which the program computes multiple different sets of inputs during the same runtime. In this case, the solution is to extract these variables such that a DP representation can be created independently of the variables that change for each iteration of the program, then reference this and add these parameters back in afterwards. An example can be seen below in my solution to `nio25-finale-belysning` under Intervals.

Longest increasing subsequence

Given a list of values, this can be used to find the longest increasing subsequence. For example in the array `[1, 4, 0, 5]`, the LIS would be `[1, 4, 5]` with a length of 3.

```
let n: usize;
let xs: Vec<i32>;

let mut dp = vec![1; n];

for i in 1..n {
  for j in 0..i {
    if xs[i] > xs[j] {
      dp[i] = dp[i].max(dp[j] + 1);
    }
  }
}

let result: i32 = *dp.iter().max().unwrap();
```

Knapsack

Given a set of values with different weights and values, we want to find the maximum value possible within the weight bounds.

```
let weights = vec![1, 2, 3, 2];
let values = vec![8, 4, 0, 5];
let capacity = 5;

let n = weights.len();
let mut dp = vec![0; capacity + 1];

for i in 0..n {
  for w in (weights[i]..=capacity).rev() {
    dp[w] = dp[w].max(dp[w - weights[i]] + values[i]);
  }
}

let result = dp[capacity];
```

This has time complexity $O(nk)$ where n is the number of items and k is the maximum capacity.

2D DP

This is an example use of DP for finding the total number of unique paths from the top-left to the bottom-right in a grid.

```
let x = 3;
let y = 3;

let mut dp = vec![vec![0; x]; y];

for i in 0..y {
  for j in 0..x {
    if i == 0 && j == 0 {
      dp[i][j] = 1;
    } else {
      let up = if i > 0 { dp[i - 1][j] } else { 0 };
      let left = if j > 0 {dp[i][j - 1] } else { 0 };
      dp[i][j] = up + left;
    }
  }
}

let result = dp[y-1][x-1];
```

Bitmask DP

Bitmasks and bitmask DP are very useful when `n` is small, often around 20. Here is an example solution for the traveling salesman problem. Here, we have a salesman who needs to visit n cities excactly once and return to the starting city. Each

pair of cities has a distance (cost) associated with traveling between them. The goal is to find the shortest route that visits each city excactly once and returns to the starting city.

```
let n = 4;
let dist = vec![
    vec![0, 10, 15, 20],
    vec![10, 0, 35, 25],
    vec![15, 35, 0, 30],
    vec![20, 25, 30, 0],
];

let size = 1 << n;

// This DP is defined by DP[mask][target city]
// The mask is a bitmask of visited cities
let mut dp = vec![vec![i32::MAX / 2; n]; size];
// With 1 visited city and ending at 0, the total cost is 0
// (not leaving the starting city)
dp[1][0] = 0;

for mask in 1..size {
    for u in 0..n {
        if (mask & (1 << u)) == 0 {
            continue; // Skip if this city is not in the mask
        }
        for v in 0..n {
            if (mask & (1 << v)) != 0 {
                continue; // Skip v if it is already in the
mask
            }
            let next_mask = mask | (1 << v);
            dp[next_mask][v] = dp[next_mask][v].min(dp[mask]
[u] + dist[u][v]);
        }
    }
}

let mut result = i32::MAX;
for u in 1..n {
    // Return to start
    result = result.min(dp[size - 1][u] + dist[u][0]);
}

println!("{}", result); // 80
```

Trees (LIS)

One of the most common DP problems on trees is finding the largest independent set in a tree, where no two selected nodes are adjacent.

```
use std::collections::VecDeque;

let n = 5;
let edges = vec![(0, 1), (0, 2), (1, 3), (1, 4)];
// Representing the tree as a graph
let mut tree = vec![Vec::new(); n];
for (u, v) in edges {
    tree[u].push(v);
    tree[v].push(u);
}

// dp[node] = [exclude, include]
let mut dp = vec![[0, 0]; n];
let mut parent = vec![n; n];
let mut order = Vec::new();

let mut stack = VecDeque::new();
stack.push_back(0);
while let Some(u) = stack.pop_back() {
    order.push(u);
    for &v in &tree[u] {
        if v == parent[u] {
            // Skip parent nodes; only traverse downwards
            continue;
        }
        parent[v] = u;
        stack.push_back(v);
    }
}

for &u in order.iter().rev() {
    dp[u][1] = 1;
    for &v in &tree[u] {
        if v == parent[u] {
            continue;
        };
        // Exclude u
        dp[u][0] += dp[v][0].max(dp[v][1]);
        // Include u => exclude children
        dp[u][1] += dp[v][0];
    }
}

println!("{}", dp[0][0].max(dp[0][1]));
```

Intervals

Sometimes, DP is used with two values for an interval (i..j). Here is an example from my solution to `nio25-finale-belysning`:

```
use std::convert::TryInto;
use std::io;

let [n, q]: [usize; 2]; // Read input

let mut ys: Vec<usize> = Vec::new();
for _ in 0..n {
    let y: usize; // Read input
    ys.push(y);
}

let mut prices: Vec<(usize, usize)> = Vec::new();
for _ in 0..q {
    let [a, b]: [usize; 2]; // read input
    prices.push((a, b));
}
```

Precompute the optimal `y`-value for a lamp in a given interval. This ignores the actual cost completely as even though the cost matters, if we have the optimal configuration of lamps for all intervals we will end up with the same total cost just using a different combination of intervals as we still only check the optimal lamp positions for a given combination of intervals.

```
let mut lys: Vec<Vec<usize>> = vec![vec![usize::MAX; n]; n];
for i in 0..n {
    for j in 0..=i {
        let p = (j..=i)
            .map(|lx| (lx, (j..=i).map(|x| ys[x] +
lx.abs_diff(x)).max().unwrap()))
            .map(|(lx, ly)| ly - ys[lx])
            .min()
            .unwrap();
        lys[j][i] = p;
    }
}

for (a, b) in prices {
    let mut dp = vec![usize::MAX; n];

    for i in 0..n {
        for j in 0..=i {
            let mut min_cost = usize::MAX;
            for lx in j..=i {
                let cost = a + b * lys[j][i];
                min_cost = min_cost.min(cost);
            }
            let prev = if j == 0 { 0 } else { dp[j - 1] };
            dp[i] = dp[i].min(prev + min_cost);
        }
    }

    println!("{:?}", dp[n - 1]);
}
```

As i described earlier, we have to find a way to precompute a DP table before running the program repeatedldy, and extract the variables from this such that the same value can be used repeatedly.

State-based DP

In problems such as `nio24-finale-manngard`, the DP can be computed linearly by keeping track of all the different valid states. This can get a bit messy

```
let n: usize; // Read input

// NOTE: making everything 1-indexed to make DP easier
let mut c: Vec<Vec<usize>> = vec![vec![usize::MAX; 2]; n +
1]; // x, y
for i in 1..=n {
    let [f, b]: [usize; 2]; // Read input
    c[i][0] = 1000 - f;
    c[i][1] = 1000 - b;
}
let c = c;

// Each person has 3 states: dark, light, torch
// (torch also implies light).
// Note that some of the states like [0, 2] are invalid
let max = usize::MAX / 2;
let mut dp: Vec<[[usize; 3]; 3]> = vec![[[max; 3]; 3]; n + 1];
dp[0][1][1] = 0;

for i in 1..=n {
    dp[i][0][0] = dp[i - 1][1][1];
    dp[i][1][0] = dp[i - 1][2][1];
    dp[i][0][1] = dp[i - 1][1][2];
    dp[i][1][1] = dp[i - 1][2][2];

    dp[i][2][1] = dp[i][2][1]
        .min(dp[i - 1][0][1] + c[i][0])
        .min(dp[i - 1][1][1] + c[i][0])
        .min(dp[i - 1][2][1] + c[i][0])
        .min(dp[i - 1][1][2] + c[i][0])
        .min(dp[i - 1][2][2] + c[i][0]);

    dp[i][1][2] = dp[i][1][2]
        .min(dp[i - 1][1][0] + c[i][1])
        .min(dp[i - 1][1][1] + c[i][1])
        .min(dp[i - 1][2][1] + c[i][1])
        .min(dp[i - 1][1][2] + c[i][1])
        .min(dp[i - 1][2][2] + c[i][1]);

    dp[i][2][2] = dp[i][2][2]
        .min(dp[i - 1][0][0] + c[i][0] + c[i][1])
        .min(dp[i - 1][1][0] + c[i][0] + c[i][1])
        .min(dp[i - 1][0][1] + c[i][0] + c[i][1])
        .min(dp[i - 1][1][1] + c[i][0] + c[i][1])
        .min(dp[i - 1][2][1] + c[i][0] + c[i][1])
        .min(dp[i - 1][1][2] + c[i][0] + c[i][1])
        .min(dp[i - 1][2][2] + c[i][0] + c[i][1]);
}

let result = dp[n][1][1]
    .min(dp[n][2][1])
    .min(dp[n][1][2])
    .min(dp[n][2][2]);

println!("{}", result);
```

Meet in the middle

This is a problem where you are given an array of integers and a target sum S, and the goal is to determine the number of subsets whose sum equals S. Trying to solve this directly will have a time complexity of $O(2^n)$, which will be too slow for $n > 30$. Instead, we can split the array in half, generate the subset sums for each half, then combine the two:

```
use std::collections::HashMap;

fn subset_sums(xs: &[i32]) -> Vec<i32> {
    let n = xs.len();
    let mut sums = Vec::new();
    for mask in 0..(1 << n) {
        let mut sum = 0;
        for i in 0..n {
            if mask & (1 << i) != 0 {
                sum += xs[i];
            }
        }
        sums.push(sum);
    }
    sums
}

fn main() {
    let xs = vec![1, 2, 3, 4, 5];
    let target = 5;

    let n = xs.len();
    let left = &xs[0..n / 2];
    let right = &xs[n / 2..];

    let left_sums = subset_sums(left);
    let right_sums = subset_sums(right);

    let mut right_count = HashMap::new();
    for &sum in &right_sums {
        *right_count.entry(sum).or_insert(0) += 1;
    }

    let mut total = 0;
    for &sum in &left_sums {
        let complement = target - sum;
        if let Some(&count) = right_count.get(&complement) {
            total += count;
        }
    }

    println!("Subsets with sum {}: {}", target, total);
}
```

Binary trees

A binary tree can be defined with

```
#[derive(Debug)]
struct Node<T> {
    value: T,
    left: Option<Box<Node<T>>>,
    right: Option<Box<Node<T>>>,
}

impl<T> Node<T> {
    fn new(value: T) -> Self {
        Node {
            value,
            left: None,
            right: None,
        }
    }
}
```

Which can then be used like

```
let mut root = Node::new(10);

root.left = Some(Box::new(Node::new(5)));
root.right = Some(Box::new(Node::new(20)));

dbg!(root);
```

Traversing it can be done either recursively or iteratively, as follows:

```
fn inorder<T: std::fmt::Display>(node: &Option<Box<Node<T>>>)
{
    if let Some(n) = node {
        inorder(&n.left);
        print!("{} ", n.value);
        inorder(&n.right);
    }
}
```

Here is an example taken from my unoptimized solution to `nio24-finale-trebygger`:

```rust
let mut root: Node<usize> = Node::new(xs[0]);
println!("0");

for &a in &xs[1..] {
    let mut h = 1;
    let mut c = &mut root;

    loop {
        if a < c.value {
            match c.left {
                Some(ref mut l) => {
                    c = l;
                }
                None => {
                    c.left = Some(Box::new(Node::new(a)));
                    println!("{}", h);
                    break;
                }
            }
        } else {
            match c.right {
                Some(ref mut r) => {
                    c = r;
                }
                None => {
                    c.right = Some(Box::new(Node::new(a)));
                    println!("{}", h);
                    break;
                }
            }
        }

        h += 1;
    }
}
```

Note

One usually will not need to implement a tree in CP - most of the time it will suffice to use a `BTreeSet` or `BTreeMap`. The solution listed above exists as an example of how one *could* use this data structure, but in this and most other cases it will be better to use one of the aforementioned data types. The above solution could be significantly optimized to this:

```rust
let mut tree: BTreeMap<usize, usize> = BTreeMap::new();

tree.insert(xs[0], 0);
println!("0");

for &x in &xs[1..] {
    let l = tree.range(..x).next_back().map(|(_, &d)| d);
    let r = tree.range(x + 1..).next().map(|(_, &d)| d);

    let d = l.unwrap_or(0).max(r.unwrap_or(0)) + 1;
    tree.insert(x, d);

    println!("{}", d);
}
```

Fenwick Trees

A Fenwick tree / binary indexed tree is a tree structure which makes handling prefix sums more efficient. Adding a value or querying a prefix are both $O(\log n)$.

```rust
struct FenwickTree {
    bit: Vec<i32>,
    n: usize,
}

impl FenwickTree {
    fn new(n: usize) -> Self {
        Self {
            bit: vec![0; n + 1],
            n,
        }
    }

    fn update(&mut self, mut i: usize, val: i32) {
        i += 1;
        while i <= self.n {
            self.bit[i] += val;
            i += i & (!i + 1).wrapping_neg();
        }
    }

    fn query(&self, mut i: usize) -> i32 {
        i += 1;
        let mut sum = 0;
        while i > 0 {
            sum += self.bit[i];
            i -= i & (!i + 1).wrapping_neg();
        }
        sum
    }
}

fn main() {
    let xs = vec![1, 2, 3, 4, 5];
    let mut ft = FenwickTree::new(xs.len());
    for (i, &val) in xs.iter().enumerate() {
        ft.update(i, val);
    }

    println!("{}", ft.query(2)); // first 3 elements: 1 + 2 + 3 = 6
    ft.update(1, 5); // Add 5 to xs[1]
    println!("{}", ft.query(2)); // first 3 elements: 1 + 7 + 3 = 11

    // Sum of range (1..=4)
    println!("{}", ft.query(4) - ft.query(0)); // 7 + 3 + 4 + 5
}
```

Segment Trees

Segment trees are a type of binary trees which can efficiently compute range queries (sum, min, max, gcd, etc), perform point updates and sometimes also range updates.

The time complexity is:

- Build: $O(n)$
- Query: $O(\log n)$
- Update: $O(\log n)$

Segment trees are like fenwick trees but much more flexible. Here is a simple segment tree implementation in rust:

```rust
struct SegmentTree {
    n: usize,
    tree: Vec<i32>,
}

impl SegmentTree {
    fn new(xs: &[i32]) -> Self {
        let n = xs.len();
        let mut tree = vec![0; 2 * n];

        for i in 0..n {
            tree[n + i] = xs[i];
        }
        for i in (1..n).rev() {
            tree[i] = tree[2 * i] + tree[2 * i + 1];
        }

        Self { n, tree }
    }

    fn update(&mut self, mut i: usize, val: i32) {
        i += self.n;
        self.tree[i] = val;
        while i > 1 {
            i /= 2;
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1];
        }
    }
```

```
    fn query(&self, mut l: usize, mut r: usize) -> i32 {
        l += self.n;
        r += self.n;
        let mut sum = 0;

        while l ≤ r {
            if l % 2 == 1 {
                sum += self.tree[l];
                l += 1;
            }
            if r % 2 == 0 {
                sum += self.tree[r];
                r -= 1;
            }
            l /= 2;
            r /= 2;
        }

        sum
    }
}

fn main() {
    let xs = vec![1, 2, 3, 4, 5];
    let mut st = SegmentTree::new(&xs);

    // Sum of range (1..=3)
    println!("{}", st.query(1, 3)); // 2 + 3 + 4
    st.update(2, 10); // xs[2] = 10
    println!("{}", st.query(1, 3)); // 2 + 10 + 4
}
```

The important part here is the `query`-function. It can be changed to anything, for example `.min()`, `.max()`, etc. Here is a more generic implementation which supports this:

```
struct SegmentTree<T, F>
where
    T: Copy,
    F: Fn(T, T) -> T,
{
    n: usize,
    tree: Vec<T>,
    combine: F,
    identity: T,
}

impl<T, F> SegmentTree<T, F>
where
    T: Copy,
    F: Fn(T, T) -> T,
{
    fn new(xs: &[T], identity: T, combine: F) -> Self {
        let n = xs.len();
        let mut tree = vec![identity; 2 * n];

        for i in 0..n {
            tree[n + i] = xs[i];
        }
        for i in (1..n).rev() {
            tree[i] = combine(tree[2 * i], tree[2 * i + 1]);
        }

        Self {
            n,
            tree,
            combine,
            identity,
        }
    }

    fn update(&mut self, mut i: usize, val: T) {
        i += self.n;
        self.tree[i] = val;
        while i > 1 {
            i /= 2;
            self.tree[i] = (self.combine)(self.tree[2 * i],
self.tree[2 * i + 1]);
        }
    }

    fn query(&self, mut l: usize, mut r: usize) -> T {
        l += self.n;
        r += self.n;

        let mut res_l = self.identity;
        let mut res_r = self.identity;

        while l ≤ r {
            if l % 2 == 1 {
                res_l = (self.combine)(res_l, self.tree[l]);
                l += 1;
            }
            if r % 2 == 0 {
                res_r = (self.combine)(self.tree[r], res_r);
                r -= 1;
            }
            l /= 2;
            r /= 2;
        }

        (self.combine)(res_l, res_r)
    }
}
```

With this, one can use different types of operations. The previous functionality can be obtained with

```
let mut st = SegmentTree::new(&xs, 0, |a, b| a + b);
```

One could also find the GCD with:

```
fn gcd(mut a: i32, mut b: i32) -> i32 {
    while b ≠ 0 {
        let t = b;
        b = a % b;
        a = t;
    }
    a
}

let mut st = SegmentTree::new(&xs, 0, gcd);
```

Frequency:

```
let xs = vec![1,0,1,1,0,1];
let mut st = SegmentTree::new(&xs, 0, |a , b| a + b);
```

Here is an overview of other operations one could do:

Query type	Comnbine function	Identity
Sum	$a + b$	0
Minimum	$\min(a, b)$	∞
Maximum	$\max(a, b)$	$-\infty$
XOR	`a ^ b`	0
GCD	$\gcd(a, b)$	0
LCM	$\text{lcm}(a, b)$	1
AND	$a \,\&\, b$	$!\,0$
OR	$a \mid b$	0
Product	$a * b$	1
Count / Frequency	$a + b$	0
Boolean OR	$a \mid\mid b$	false
Boolean AND	$a \,\&\&\, b$	true

For any segment tree operation, it must be associative. This means that it must satisfy

$$f(a, f(b, c)) = f(f(a, b), c)$$

It must also have an identity which satisfies

$$f(\text{identity}, x) = x$$

If both of these apply, it is possible to use it in a segment tree. One can also create a `Node`-struct which stores multiple of these values at the same time. It can for example be used as such:

```
#[derive(Clone, Copy)]
struct Node {
    sum: i32,
    min: i32,
    max: i32,
}

impl Node {
    fn new(x: i32) -> Self {
        Self {
            sum: x,
            min: x,
            max: x,
        }
    }

    fn identity() -> Self {
        Self {
            sum: 0,
            min: i32::MAX,
            max: i32::MIN,
        }
    }

    fn combine(a: Self, b: Self) -> Self {
        Self {
            sum: a.sum + b.sum,
            min: a.min.min(b.min),
            max: a.max.max(b.max),
        }
    }
}

struct SegmentTree {
    n: usize,
    tree: Vec<Node>,
}

impl SegmentTree {
    fn new(xs: &[i32]) -> Self {
        let n = xs.len();
        let mut tree = vec![Node::identity(); 2 * n];

        for i in 0..n {
            tree[n + i] = Node::new(xs[i]);
        }
        for i in (1..n).rev() {
            tree[i] = Node::combine(tree[2 * i], tree[2 * i +
1]);
        }

        Self { n, tree }
    }

    fn update(&mut self, mut i: usize, val: i32) {
        i += self.n;
        self.tree[i] = Node::new(val);

        while i > 1 {
            i /= 2;
            self.tree[i] = Node::combine(self.tree[2 * i],
self.tree[2 * i + 1]);
        }
    }

    fn query(&self, mut l: usize, mut r: usize) -> Node {
        l += self.n;
        r += self.n;

        let mut res_l = Node::identity();
        let mut res_r = Node::identity();

        while l <= r {
            if l % 2 == 1 {
                res_l = Node::combine(res_l, self.tree[l]);
                l += 1;
            }
            if r % 2 == 0 {
                res_r = Node::combine(self.tree[r], res_r);
                r -= 1;
            }
            l /= 2;
            r /= 2;
        }

        Node::combine(res_l, res_r)
    }
}
```

Note

I have just hard-coded the `i32`-types as it becomes an unnecessary amount of complexity if i were to make everything generic, which is generally useless for competitive programming. Again, this can store the results of *any* associative operation which also has an identity.

Lazy propagation

A segment tree can be extended to allow range updates efficiently. With this tree, the time complexity is

- Build: $O(n)$
- Range update: $O(\log n)$
- Range query: $O(\log n)$

```
struct SegmentTree {
    n: usize,
    tree: Vec<Node>,
    lazy: Vec<i32>,
}

impl SegmentTree {
    fn new(xs: &[i32]) -> Self {
        let n = xs.len();
        let mut seg = Self {
            n,
            tree: vec![Node::identity(); 4 * n],
            lazy: vec![0; 4 * n],
        };

        seg.build(1, 0, n - 1, xs);
        seg
    }

    fn build(&mut self, v: usize, l: usize, r: usize, xs:
&[i32]) {
        if l == r {
            self.tree[v] = Node::new(xs[l]);
            return;
        }

        let mid = (l + r) / 2;
        self.build(v * 2, l, mid, xs);
        self.build(v * 2 + 1, mid + 1, r, xs);

        self.tree[v] = Node::combine(self.tree[v * 2],
self.tree[v * 2 + 1]);
    }

    fn push(&mut self, v: usize, l: usize, r: usize) {
        let lazy_val = self.lazy[v];
        if lazy_val == 0 {
            return;
        }

        let mid = (l + r) / 2;
        let left = v * 2;
        let right = v * 2 + 1;

        self.tree[left].apply(lazy_val, (mid - l + 1) as i32);
        self.tree[right].apply(lazy_val, (r - mid) as i32);

        self.lazy[left] += lazy_val;
        self.lazy[right] += lazy_val;

        self.lazy[v] = 0;
    }
```

```
    fn update_range(&mut self, v: usize, l: usize, r: usize,
ql: usize, qr: usize, val: i32) {
        if ql > r || qr < l {
            return;
        }

        if ql <= l && r <= qr {
            self.tree[v].apply(val, (r - l + 1) as i32);
            self.lazy[v] += val;
            return;
        }

        self.push(v, l, r);

        let mid = (l + r) / 2;

        self.update_range(v * 2, l, mid, ql, qr, val);
        self.update_range(v * 2 + 1, mid + 1, r, ql, qr, val);

        self.tree[v] = Node::combine(self.tree[v * 2],
self.tree[v * 2 + 1]);
    }

    fn query(&mut self, v: usize, l: usize, r: usize, ql:
usize, qr: usize) -> Node {
        if ql > r || qr < l {
            return Node::identity();
        }

        if ql <= l && r <= qr {
            return self.tree[v];
        }

        self.push(v, l, r);

        let mid = (l + r) / 2;

        let left = self.query(v * 2, l, mid, ql, qr);
        let right = self.query(v * 2 + 1, mid + 1, r, ql, qr);

        Node::combine(left, right)
    }

    fn update(&mut self, l: usize, r: usize, val: i32) {
        self.update_range(1, 0, self.n - 1, l, r, val);
    }

    fn range_query(&mut self, l: usize, r: usize) -> Node {
        self.query(1, 0, self.n - 1, l, r)
    }
}
```

```
fn main() {
    let xs = vec![1, 3, 5, 7, 9, 11];
    let mut st = SegmentTree::new(&xs);
    st.update(1, 4, 10);
    let res = st.range_query(0, 5);
    println!("sum = {}", res.sum);
    println!("min = {}", res.min);
    println!("max = {}", res.max);
}
```

DSU / Union-Find

A Disjoint Set Union is a data structure that can keep track of multiple disjoint / non-overlapping sets, and can efficiently do the following operations:

- Find: Determine which set an element belongs to
- Union: Merge two sets into one

This is useful to track connectivity. An example implementation in rust is as follows:

```
struct DSU {
    parent: Vec<usize>,
    size: Vec<usize>,
}

impl DSU {
    fn new(n: usize) -> Self {
        DSU {
            parent: (0..n).collect(),
            size: vec![1; n],
        }
    }

    fn find(&mut self, x: usize) -> usize {
        if self.parent[x] != x {
            // Compress the path with recursion
            self.parent[x] = self.find(self.parent[x]);
        }
        self.parent[x]
    }

    // False if the sets are already the same
    fn union(&mut self, x: usize, y: usize) -> bool {
        let mut rx = self.find(x);
        let mut ry = self.find(y);
        if rx == ry {
            return false;
        }
        // Union the larger of the two sets
        if self.size[rx] < self.size[ry] {
            std::mem::swap(&mut rx, &mut ry);
        }
        self.parent[ry] = rx;
        self.size[rx] += self.size[ry];
        true
    }
}
```

Note

The time complexity of both the `find` and `union` methods in a `DSU` both have the time complexity $O(\alpha(n))$ where α is the inverse Ackermann function. In practice, $\alpha(n) \leq 5$ for all reasonable values of n, so it can be treated as constant.

This can be a bit confusing to think about, but here is an example:

```
let mut dsu = DSU::new(5);
// Has 5 sets: {0}, {1}, {2}, {3}, {4}

dsu.union(0, 1);
// Has 4 sets: {0, 1}, {2}, {3}, {4}
dsu.union(1, 2);
// Has 3 sets: {0, 1, 2}, {3}, {4}
dsu.union(3, 4);
// Has 2 sets: {0, 1, 2}, {3, 4}
dsu.union(1, 3)
// Has 1 set: {0, 1, 2, 3, 4}
```

I used this in my solution for `nio24-runde2-tognett`. Note that this solution is not optimal, and only gave `65/100` points, however this is still much more efficient than one which uses BFS for each iteration.

```
let mut dsu = DSU::new(n);

for &(a, b) in &trains {
    dsu.union(a, b);

    let mut removed = 0;
    for u in 0..n {
        let mut to_remove = Vec::new();
        for &v in &plane_graph[u] {
            if dsu.find(u) == dsu.find(v) {
                to_remove.push(v);
            }
        }
        removed += to_remove.len();
        for v in to_remove {
            plane_graph[u].remove(&v);
        }
    }

    println!("{}", removed / 2);
}
```

This is very useful in cases where one wants to find the connected components in a graph. Instead of running a BFS search from one node, one can use a DSU to store it. Using that in this task would be $O(k(n+m))$, while using a DSU has the same time complexity but is much faster in practice because the overhead of using a BFS and queue is avoided.

Difference arrays

Difference arrays provide a more efficient way to update all values within an interval. Instead of looping over all items in the interval $[a, b]$, one instead just sets the values at a and b, then passes over the array afterwards. A boolean implementation could look like this (taken from my solution to `nio23-runde2-lynnedslag`):

```
let mut houses: Vec<bool> = vec![false; n];

for _ in k {
  let [a, b]: [usize; 2];
  houses[a] = !houses[a];
  houses[b+1] = !houses[b+1];
}

let mut flip = false;
let mut result = 0;

for h in houses {
  flip ^= h;

  if !flip {
    result += 1;
  }
}
```

Note

We use $b+1$ instead of b because the interval is inclusive. The difference array marks where the effect starts and stops, and the prefix accumulation applies the effect from a through b.

A more general implementation could look like this:

```
let mut xs: Vec<isize> = vec![0; 10 + 1];

// Add 3 to the interval [2, 5]
xs[2] += 3;
xs[5+1] -= 3;

// Subtract 9 in the interval [6, 9]
xs[6] -= 9;
xs[9+1] += 9;

let mut d = 0;
let mut result = 0;

for x in xs {
  d += x;
  result += d;
}
```

Note that the previous version is actually a special case of this, in which one would use `+= x % 2` (which I have simplified to using booleans).

Various other algorithms

Binary search

Given a sorted list, one can find a value in $O(\log n)$ using binary search. This also works for any monotonic function in which one wants to find the value in which it turns, and is thus useful in many problems. Here is an example of binary search implemented with a monotonic boolean function which starts at false and then becomes true, like `[false, false, false, true, true]`.

```
let mut a: usize = 0;
let mut b: usize = n; // n is the max value
let mut result: Option<usize> = None;

while a <= b {
  let mid = (a + b) / 2;
  if some_predicate(mid) {
    result = Some(mid);
    b = mid - 1;
  } else {
    a = mid + 1;
  }
}
```

Probability

One of the tasks usually contain some sort of probability, usually some sort of linear distribution in an interval. Here is my solution for `nio25-finale-jobbjakt`:

Vi er ikke gitt noen globale variabler, men dette er et sannsynlighetsproblem der vi vil fokusere på N. Her definerer jeg N som antall jobbtilbud igjen (altså starter man for eksempel på $N = 4$, så $N = 3$, $N = 2$ etc). Jeg inkluderer først de lokale variablene l og h for å løse likningen, men setter disse lik henholdsvis 0 og 1, slik at jeg kan sette inn igjen disse variablene etterpå.

Jeg ønsker å fine den forventede verdien for lønn med en gitt N. Hvis et gitt tilbud er høyere enn forventningsverdien, skal jeg velge å akseptere det. Jeg starter da ved å definere grunntilstanden $E_0 = 0$, da man er nødt til å akseptere enhver verdi hvis det ikke er noen tilbud igjen. Videre har jeg at den forventede verdien for $N = 1$ er $E_1 = \frac{h+l}{2}$. For å forenkle dette substituerer jeg l og h som beskrevet og får da at $E_1 = 0.5$.

Når $N = 2$, bruker man E_1 som terskel på tilbudet. Hvis tilbudet er $< E_1$, takker man nei. Forventningsverdien hvis man takker nei blir da det samme som situasjonen der $N = 1$, altså E_1. Hvis man takker ja derimot, har vi at tilbudet $> E_1$. Dden forventede verdien ligger da i intervallet $[E_1, h]$. Da dette er en uniform distribusjon har vi nå at den forventede verdien er $\frac{E_1+h}{2}$, som da blir $\frac{E_1+1}{2}$. Sannsynligheten for at tilbudet ligger i intervallet blir da $1 - P_{\text{nei}}$ $(1 - E_1)$, slik at summen er 1. Hvis vi setter inn verdien for E_1 får vi da 0.75. Den samlede forventningsverdien er gitt ved

$$E_2 = \underbrace{0.5}_{P_{\text{nei}}} \cdot \underbrace{0.5}_{E_{\text{nei}}} + \underbrace{(1-0.5)}_{P_{\text{ja}}} \cdot \underbrace{\frac{0.5+1}{2}}_{E_{\text{ja}}} = 0.625$$

Jeg kan trekke den samme logikken videre for å beregne E_3:

$$E_3 = \underbrace{0.625}_{P_{\text{nei}}} \cdot \underbrace{0.625}_{E_{\text{nei}}} + \underbrace{(1-0.625)}_{P_{\text{ja}}} \cdot \underbrace{\frac{0.625+1}{2}}_{E_{\text{ja}}}$$

Bruker denne logikken til å skrive det generelle uttrykket:

$$E_2 = \underbrace{E_{n-1}}_{P_{\text{nei}}} \cdot \underbrace{E_{n-1}}_{E_{\text{nei}}} + \underbrace{(1-E_{n-1})}_{P_{\text{ja}}} \cdot \underbrace{\frac{E_{n-1}+1}{2}}_{E_{\text{ja}}}$$

Og forenkler dette:

$$\begin{aligned} E_n &= E_{n-1}^2 + \frac{1}{2} \cdot (1 - E_{n-1}) \cdot (E_{n-1} + 1) \\ &= E_{n-1}^2 + \frac{1}{2} \cdot (E_{n-1} + 1 - E_{n-1}^2 - E_{n-1}) \\ &= \frac{2E_{n-1}^2}{2} + \frac{-E_{n-1}^2 + 1}{2} \\ E_n &= \frac{1 + E_{n-1}^2}{2} \end{aligned}$$

Der n er antall tilbud igjen etter dette tilbudet. Dette kan implementeres i rust ved hjelp av DP med tabulation som følger:

```rust
use std::io;

fn main() {
    let mut lines = io::stdin().lines();

    const N_MAX: usize = 20;
    let mut dp: Vec<f32> = vec![0.0; N_MAX + 1];
    dp[0] = 0.0;
    dp[1] = 0.5;
    for n in 2..=N_MAX {
        dp[n] = (1.0 + dp[n - 1] * dp[n - 1]) / 2.0;
    }

    let r: usize =
lines.next().unwrap().unwrap().parse().unwrap();
    for _ in 0..r {
        let line = lines.next().unwrap().unwrap();
        let mut iter = line.trim().split_whitespace();
        let n: usize = iter.next().unwrap().parse().unwrap();
        let l: f32 = iter.next().unwrap().parse().unwrap();
        let h: f32 = iter.next().unwrap().parse().unwrap();

        for i in 0..n {
            let n_remaining = n - i - 1;
            let v: f32 =
lines.next().unwrap().unwrap().parse().unwrap();

            if v >= l + (h - l) * dp[n_remaining] {
                println!("ja");
                break;
            } else {
                println!("nei");
            }
        }
    }
}
```

Binomial coefficient

Given a set of n values, the binomial coefficient tells us how many different ways we can choose k objects from this set, regardless of order. It is written as

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Note that $k!(n-k)!$ means that we can set k to $n-k$ and get the same result. Here is an implementation in rust taking advantage of this algorithm, which runs in $O(n)$.

```rust
fn binom(n: u64, k: u64) {
  let k = k.min(n - k);
  let mut result = 1;

  for i in 0..k {
    result = result * (n - i) / (i + 1);
  }

  result
}
```

This is the same as Pascal's triangle:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

With this, we can use DP to build a table of binomial coefficients. This essentially uses tabulation to create Pascal's triangle, then reads the value from that. The benefit to the DP method is that it also stores the previous results. `k` could be substituted with `n` to generate the full triangle and be able to get all binomial coefficients up to $\binom{n}{n}$. This algorithm is $O(nk)$.

```rust
let mut c = vec![vec![0; k + 1]; n + 1];

for i in 0..=n {
    for j in 0..=k.min(i) {
        if j == 0 || j == i {
            c[i][j] = 1;
        } else {
            c[i][j] = c[i - 1][j - 1] + c[i - 1][j];
        }
    }
}

c[n][k]
```

Here are some common problems:

Grid path counting

Say we have a grid of size $m \times n$ and can only move right or down. In other words, one must take n down moves and m right moves. Thus, the total number of paths can be expressed as

$$\binom{m+n}{n}$$

Choosing subsets

- "From n elements, how many subsets contain excactly k elements?"
- "How many binary strings of length n contain excactly k ones?"
- "Distribute n identical items across k containers."
- "Flip a coin n times. How many ways is there to get k heads?"

$$\binom{n}{k}$$

DP

Binomial coefficients also often appear in DP. Example: "Count ways to build a structure by choosing elements step-by-step"

$$dp[n] = dp[k] \cdot \binom{n}{k}$$

Prefix Sums

A prefix sum is a new array built from an array where each element is the sum of all previous elements in the original array up to i. For example:

```
a = [2, 4, 1, 3];
prefix[0] = 2;
prefix[1] = 2 + 4;
prefix[2] = 2 + 4 + 1;
prefix[3] = 2 + 4 + 1 + 3;
prefix = [2, 6, 7, 10];
```

After the prefix sum is computet, one can find the sum of a subarray in $O(1)$:

```
sum(l..r) = prefix[r] - prefix[l-1];
```

Sieve of Eratosthenes

The sieve of Erastothenes is an efficient algorithm used to find all prime numbers up to a given `n`.

```rust
fn sieve(n: usize) -> Vec<bool> {
  let mut is_prime = vec![true; n + 1];
  if n >= 0 { is_prime[0] = false; }
  if n >= 1 { is_prime[1] = false; }

  for p in 2..=((n as f64).sqrt() as usize) {
    if is_prime[p] {
      let mut multiple = p * p;
      while multiple <= n {
        is_prime[multiple] = false;
```

```
        multiple += p;
      }
    }
  }

  is_prime
}
```

And a `Vec` of primes can be created with

```
is_prime.iter()
        .enumerate()
        .filter_map(|(i, &prime)| if prime { Some(i) } else
{ None })
        .collect();
```

Convex Hull

For a given list of points, the convex hull is the polygon with the smallest possible area while containing all points.

```
fn cross(o: (i64, i64), a: (i64, i64), b: (i64, i64)) -> i64 {
    (a.0 - o.0) * (b.1 - o.1) - (a.1 - o.1) * (b.0 - o.0)
}

fn convex_hull(mut pts: Vec<(i64, i64)>) -> Vec<(i64, i64)> {
    if pts.len() <= 1 {
        return pts;
    }
    pts.sort();

    let mut lower = vec![];
    for &p in &pts {
        while lower.len() >= 2 && cross(lower[lower.len() -
2], lower[lower.len() - 1], p) <= 0 {
            lower.pop();
        }
        lower.push(p);
    }

    let mut upper = vec![];
    for &p in pts.iter().rev() {
        while upper.len() >= 2 && cross(upper[upper.len() -
2], upper[upper.len() - 1], p) <= 0 {
            upper.pop();
        }
        upper.push(p);
    }

    lower.pop();
    upper.pop();
    lower.extend(upper);
    lower
}

fn main() {
    let points = vec![(0, 0), (1, 1), (2, 2), (2, 0), (0, 2),
(1, 0)];
    let hull = convex_hull(points);
    println!("{:?}", hull); // [(0, 0), (2, 0), (2, 2), (0,
2)]
}
```

Sliding Window / Two Pointers

Maximum subarray of size `k`:

```
let mut sum: i32 = arr.iter().take(k).sum();
let mut max_sum = sum;

for i in k..arr.len() {
    sum += arr[i] - arr[i - k];
    max_sum = max_sum.max(sum);
}
```

Find all pairs with sum `<= s` in $O(n)$:

```
arr.sort();
let mut i = 0;
let mut j = arr.len() as i32 - 1;
let mut count = 0;

while i < j as usize {
    if arr[i] + arr[j as usize] <= s {
        count += j - i as i32;
        i += 1;
    } else {
        j -= 1;
    }
}
```

Longest subarray with sum `<= k`:

```
let mut sum = 0;
let mut left = 0;
let mut max_len = 0;

for right in 0..arr.len() {
    sum += arr[right];

    while sum > k && left <= right {
        sum -= arr[left];
        left += 1;
    }

    max_len = max_len.max(right - left + 1);
}
```

Factorial

It is usually best to precompute factorials using tabulation to avoid unuecessary recomputations of the same value:

```
let mut factorial: Vec<u64> vec![0; n + 1];
factorial[0] = 1;
for i in 1..=n {
  factorial[i] = i * factorial[i - 1];
}
```

As described earlier, we usually when dealing with large numbers want do to $\mod(10^9 + 7)$:

```
const MOD: u64 = 1_000_000_007;

let mut factorial: Vec<u64> vec![0; n + 1];
factorial[0] = 1;
for i in 1..=n {
  factorial[i] = i * factorial[i - 1] % MOD;
}
```